Announcements

- PA2 (vectorization) due tomorrow 10/2, free late deadline 10/4
- Project 1 (particle simulation) due 10/12, free late deadline 10/14
- PACE ICE down for maintenance 10/5-8
- No class/OH next Tuesday 10/6 (fall break), lecture on Zoom next Thursday 10/8

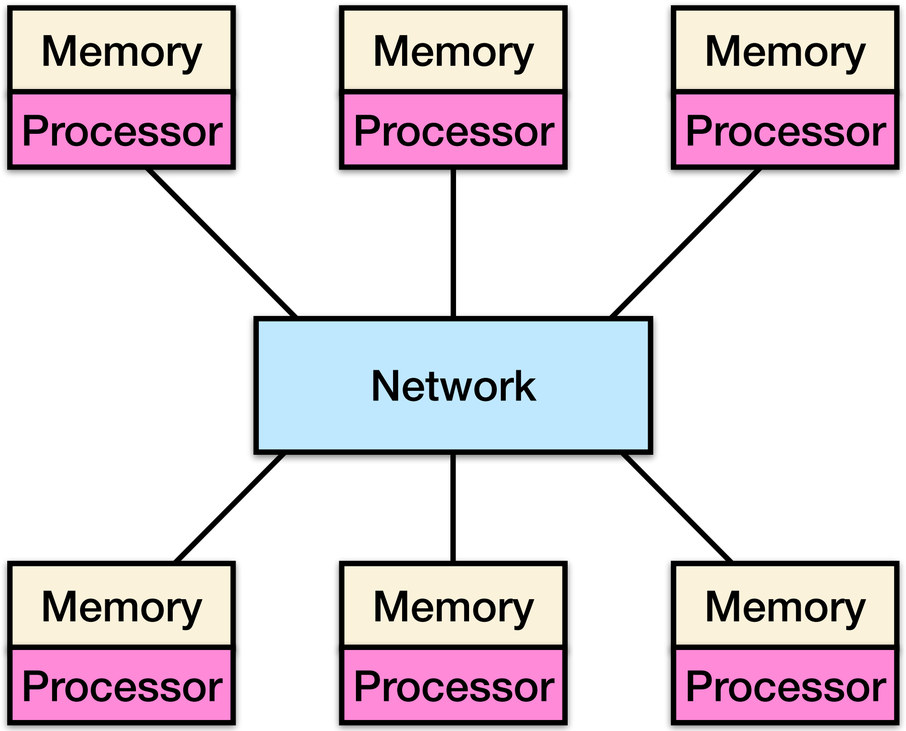
CSE 6230:
HPC Tools and Applications

Lecture 9: Distributed-Memory Programming

Helen Xu
hxu615@gatech.edu

Georgia Tech College of Computing
School of Computational Science and Engineering

Recall: Distributed-Memory Machines

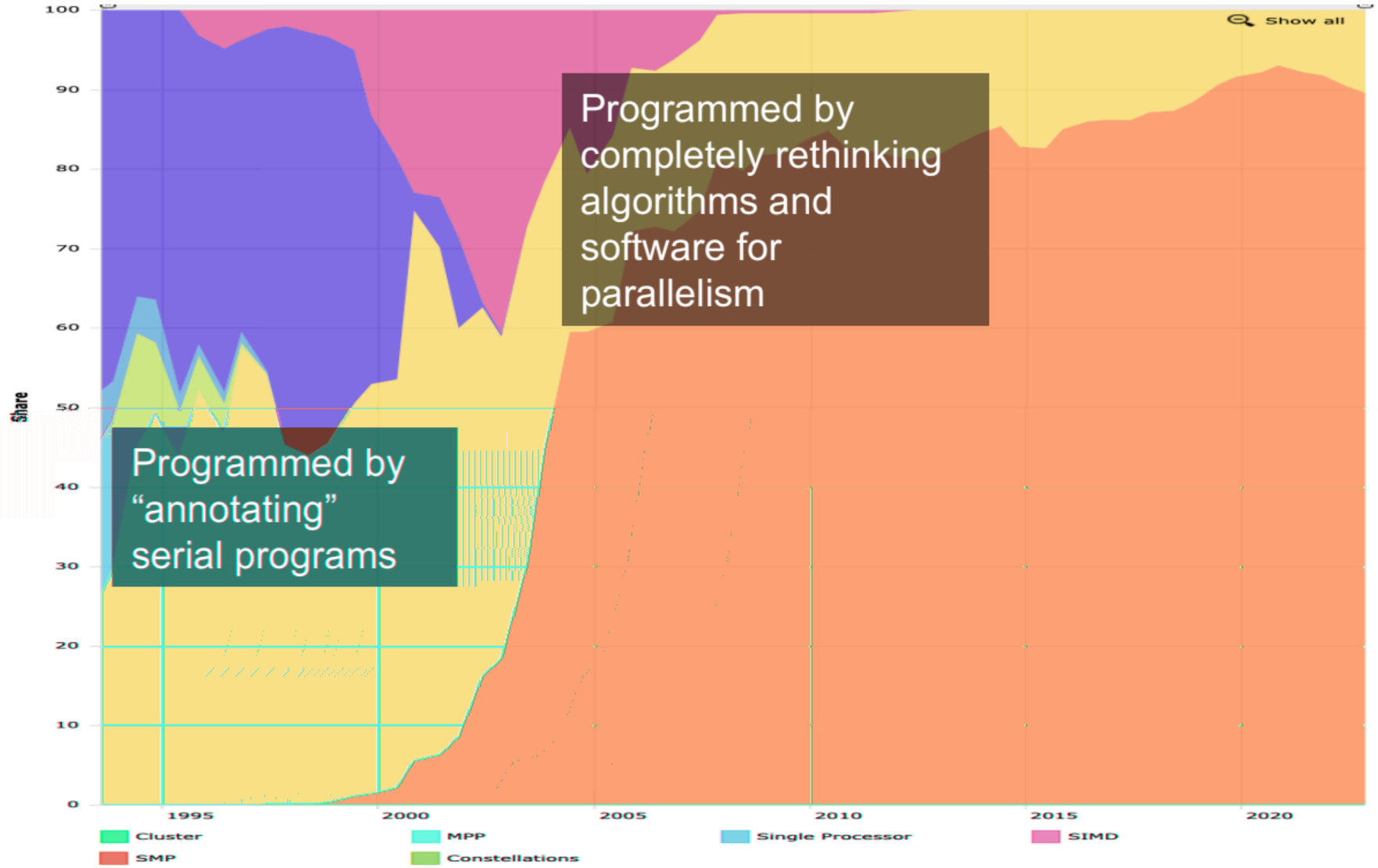


A **distributed-memory multiprocessor** has processors with their own memories connected by a high-speed network.

Also called a **cluster**.

A **high-performance computing** (HPC) system contains 100s or 1000s of such processors (nodes).

From vector machines to massively parallel accelerator systems



From Ramki Kannan

Example: Frontier System

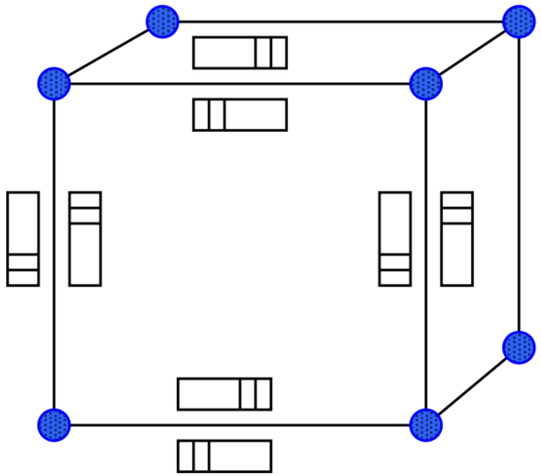
System	Titan (2012)	Summit (2017)	Frontier (2021)
Peak	27 PF	200 PF	2 EF
# nodes	18,688	4,608	9,408
Node	1 AMD Opteron CPU 1 NVIDIA Kepler GPU	2 IBM POWER9™ CPUs 6 NVIDIA Volta GPUs	1 AMD EPYC CPU 4 AMD Radeon Instinct GPUs
Memory	0.6 PB DDR3 + 0.1 PB GDDR5	2.4 PB DDR4 + 0.4 HBM + 7.4 PB On-node storage	4.6 PB DDR4 + 4.6 PB HBM2e + 66 PB On-node storage, 60 TB/s Read 62 TB/s Write
On-node interconnect	PCI Gen2 No coherence across the node	NVIDIA NVLINK Coherent memory across the node	AMD Infinity Fabric Coherent memory across the node
System Interconnect	Cray Gemini network 6.4 GB/s	Mellanox Dual-port EDR IB 25 GB/s	Four-port Slingshot network 100 GB/s
Topology	3D Torus	Non-blocking Fat Tree	Dragonfly
Storage	32 PB, 1 TB/s, Lustre Filesystem	250 PB, 2.5 TB/s, IBM Spectrum Scale™ with GPFS™	695 PB HDD+11 PB Flash Performance Tier, 9.4 TB/s and 10 PB Metadata Flash. Lustre
Power	9 MW	13 MW	29 MW

Report on ORNL Frontier System – Jack Donggarra and Al Geist https://icl.utk.edu/files/publications/2022/icl-utk-1570-2022.pdf

Distributed-Memory Architectures

Historical Perspective

- Early distributed memory machines were:
  - Collection of microprocessors.
  - Communication was performed using bi-directional **queues between nearest neighbors**.
- Messages were **forwarded by processors** on path - “Store and forward” networking
- There was a strong emphasis on **topology in algorithms**, in order to minimize the number of hops = minimize time



Network Analogy

To have a large number of different transfers occurring at once, you need a large number of distinct wires

Not just a bus, as in shared memory

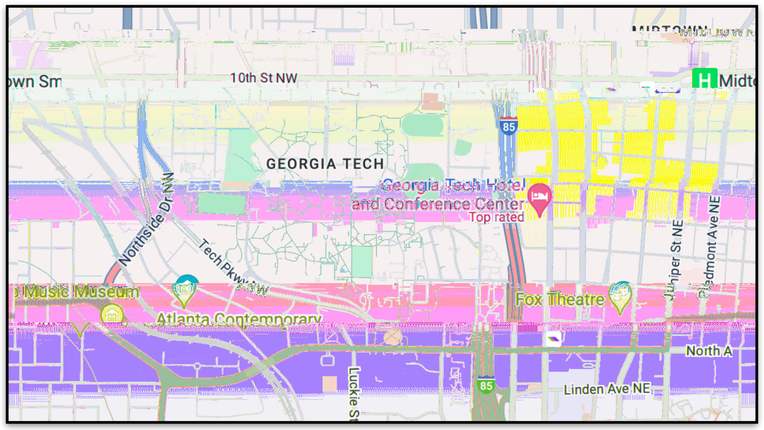
Networks are like streets:

Link = street.

Switch = intersection.

Distances (hops) = number of blocks traveled.

Routing algorithm = travel plan.



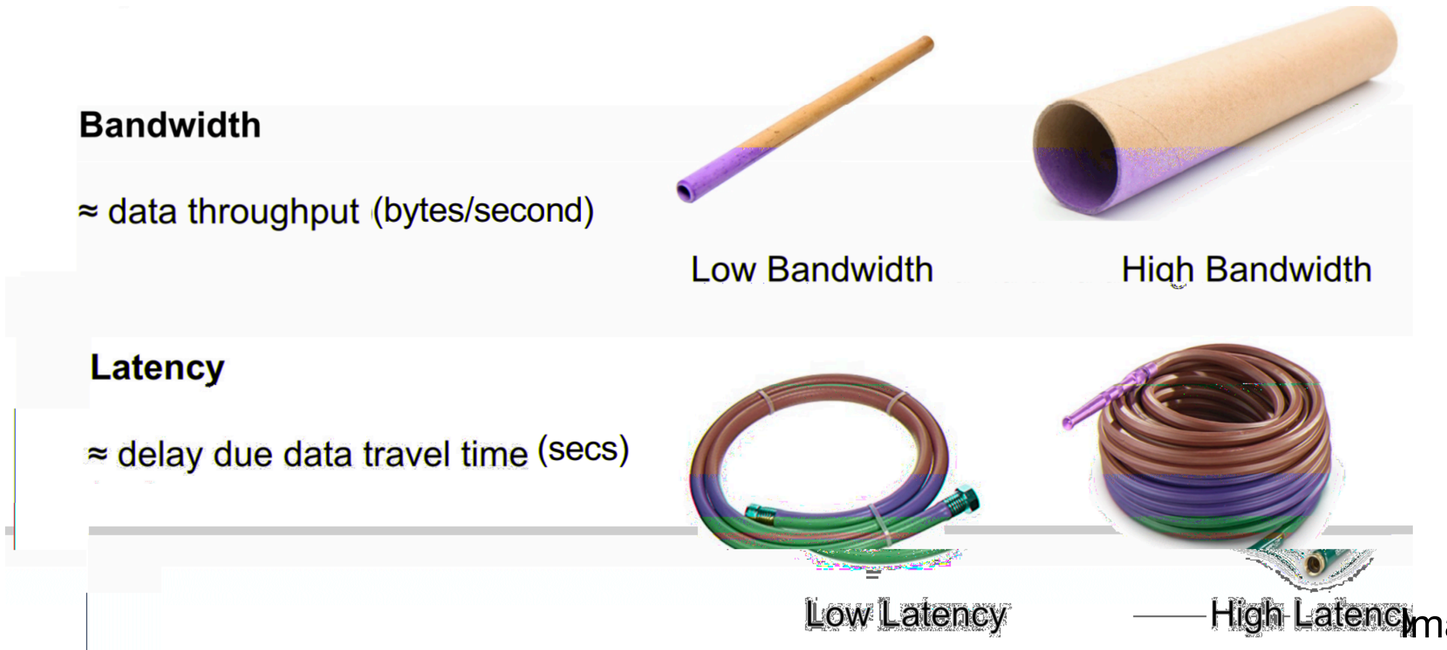
Network Properties

Latency - how long to get between nodes in the network

- Street - time for one car = dist (miles) / speed (miles/hr)

Bandwidth - how much data can be moved per unit time

- Street - cars / hour = density (cars / mile) * speed (miles / hr) * #lanes
- Network bandwidth is limited by the bit rate per wire and #wires

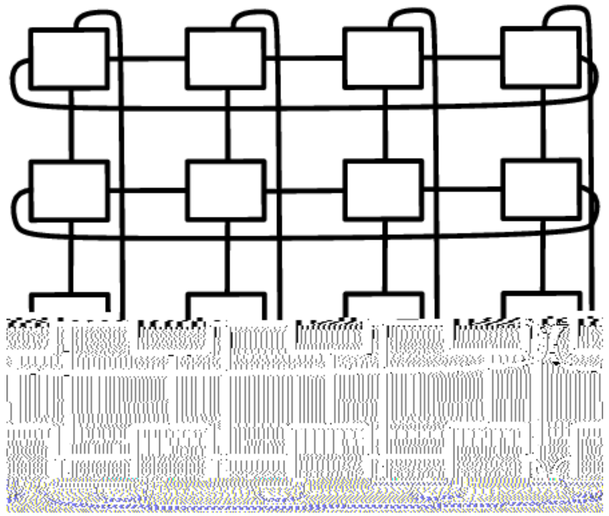


Network Design Characteristics

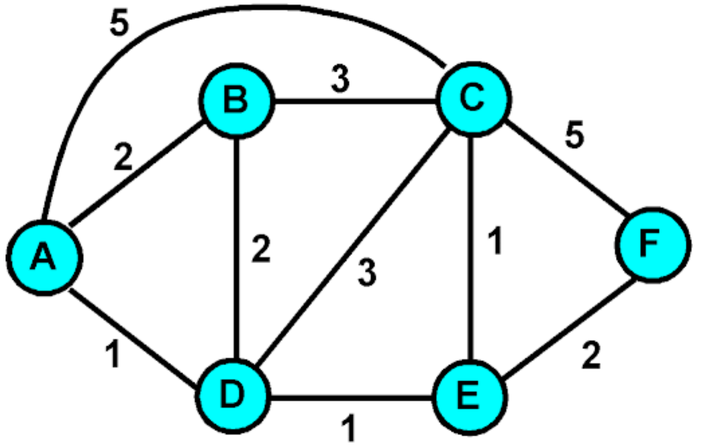
Topology (how things are connected) - examples include crossbar; ring; 2-D, 3-D, higher-D mesh or torus; hypercube; tree; butterfly; perfect shu e, dragon fly, …

Routing algorithm - how messages move through the network

Example in 2D torus: all east-west then all north-south (avoids deadlock).



https://www.researchgate.net/figure/A-4-4-2D-torus-network_fig12_350954542



http://www2.ic.uff.br/~michael/kr1999/4-network/4_02-algor.htm

Performance Properties: Latency

Diameter: the maximum (over all pairs of nodes) of the shortest path between a given pair of nodes.

Latency: delay between send and receive times

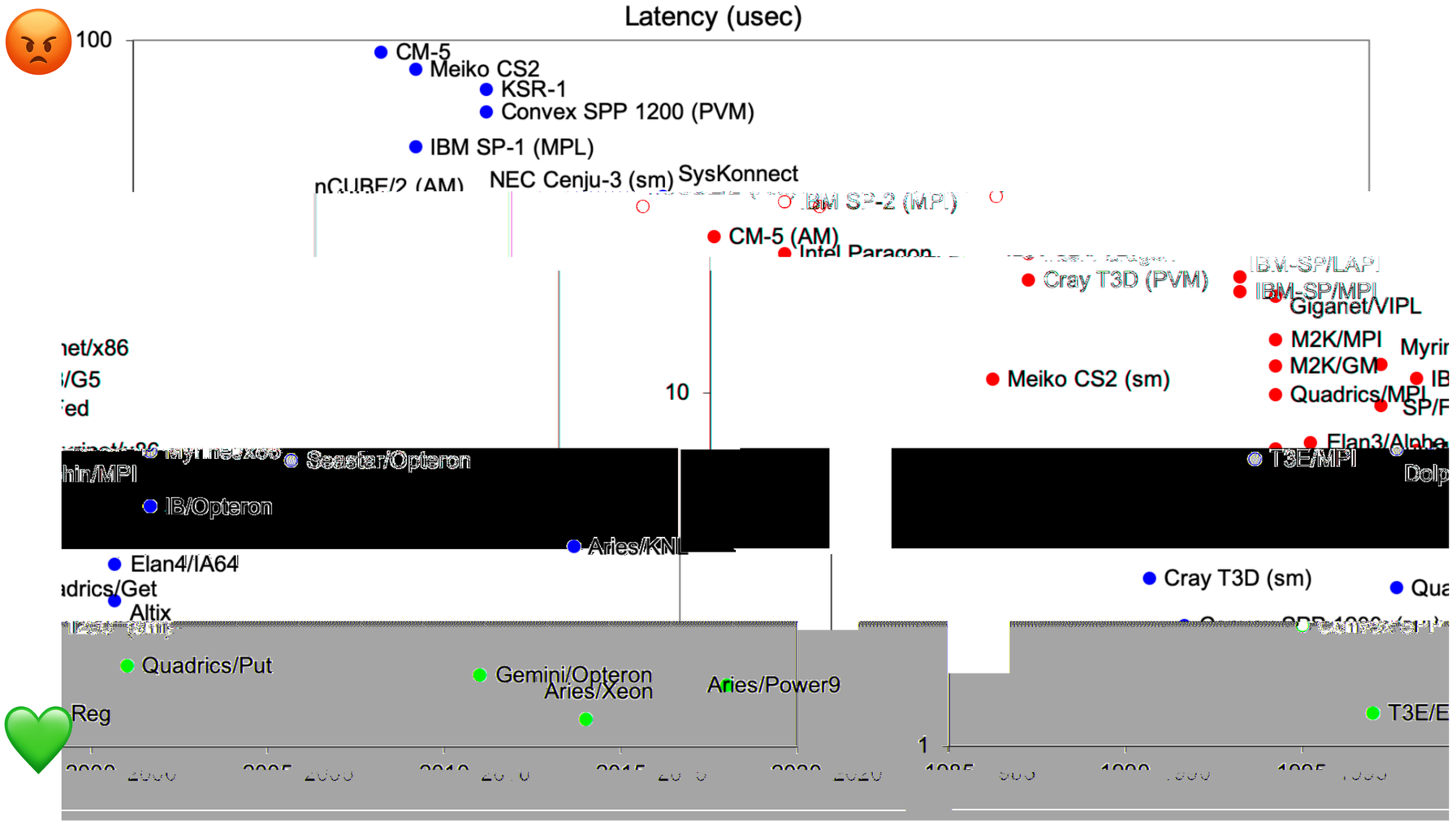
- Vendors often report **hardware latencies** (wire time).
- Application programmers care about **software latencies** (user program to user program).

Software/hardware overhead at source/destination dominate cost (1s-10s usecs).

- Hardware latency varies with distance (10s-100s nsec per hop) but is small compared to overheads.

Latency is key for programs **with many small messages.**

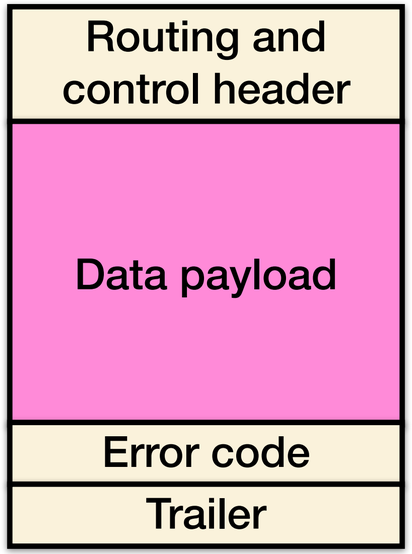
End-to-End Latency (1/2 Roundtrip) Over Time



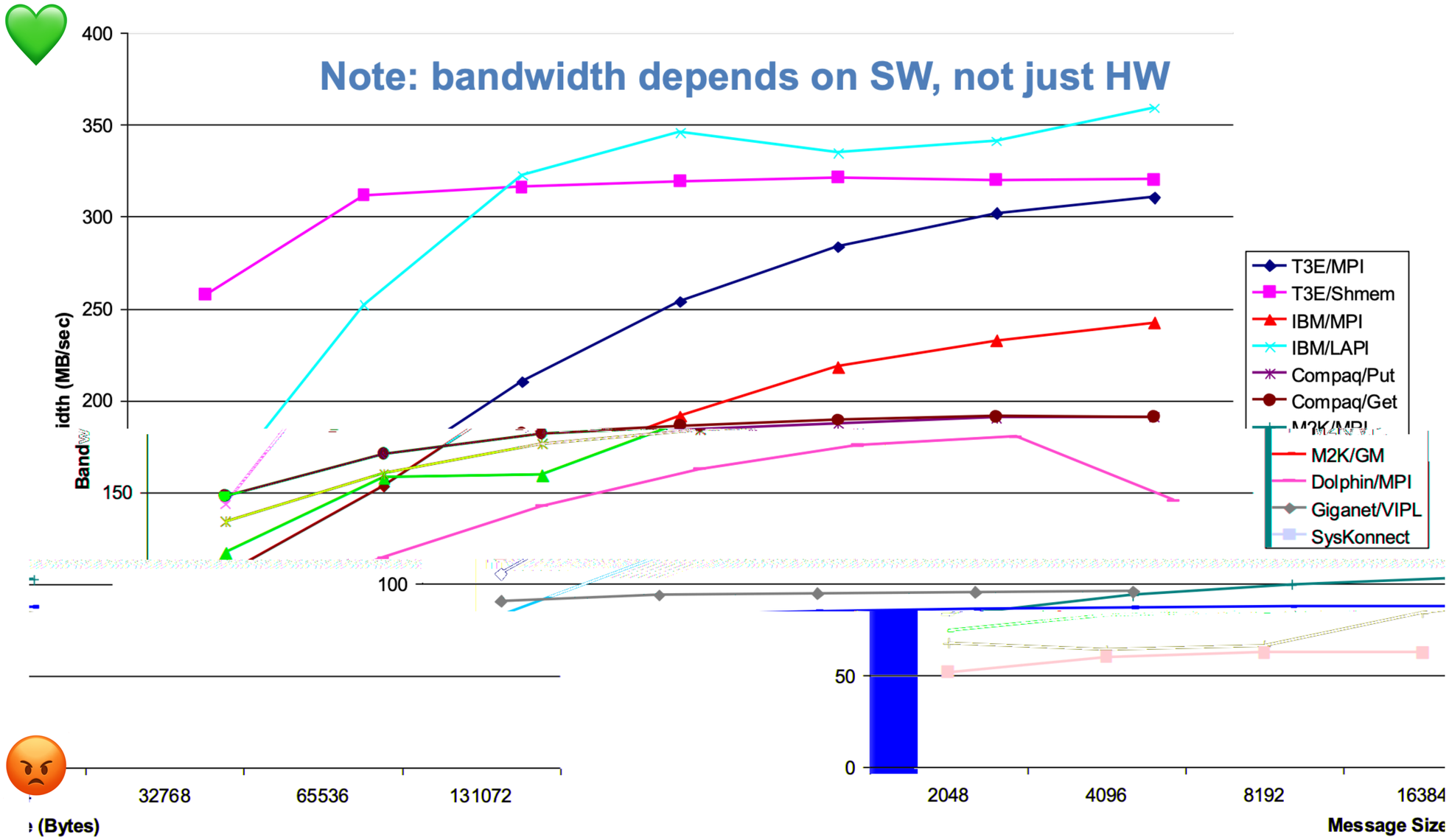
Latency has not improved as much as transistor counts (i.e., Moore's Law)

Performance Properties: Bandwidth

- The bandwidth of a link = # wires / time-per-bit
- Bandwidth typically in **Gigabytes/sec** (GB/s)
- **Effective bandwidth is usually lower** than physical link bandwidth due to packet overhead.
- Bandwidth is important for applications with mostly **large messages.**



Bandwidth Performance

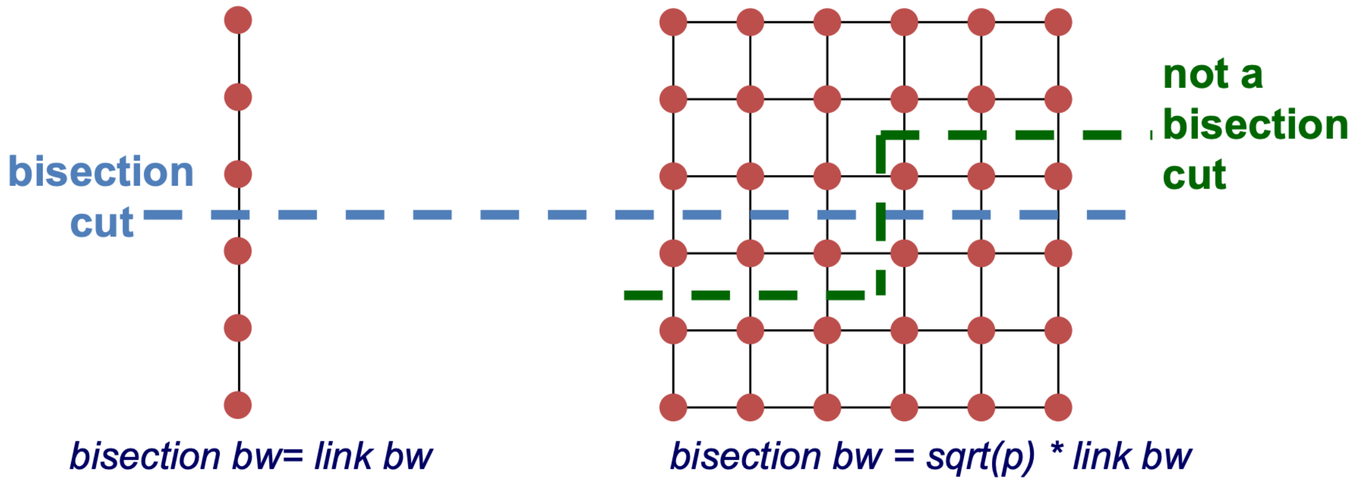


Data from Mike Welcome, NERSC

Performance Properties: Bisection Bandwidth

Bisection bandwidth: **bandwidth across smallest cut** that divides network into two equal halves

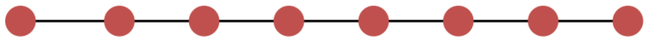
Bandwidth across “narrowest” part of the network



Bisection bandwidth is important for algorithms in which **all processors need to communicate with all others**.

Linear and Ring Topologies

Linear array



- Diameter = n-1; average distance ~n/3.
- Bisection bandwidth = 1 (in units of link bandwidth).

Torus or Ring



- Diameter = n/2; average distance ~ n/4.
- Bisection bandwidth = 2.
- Natural for algorithms that work with 1D arrays.

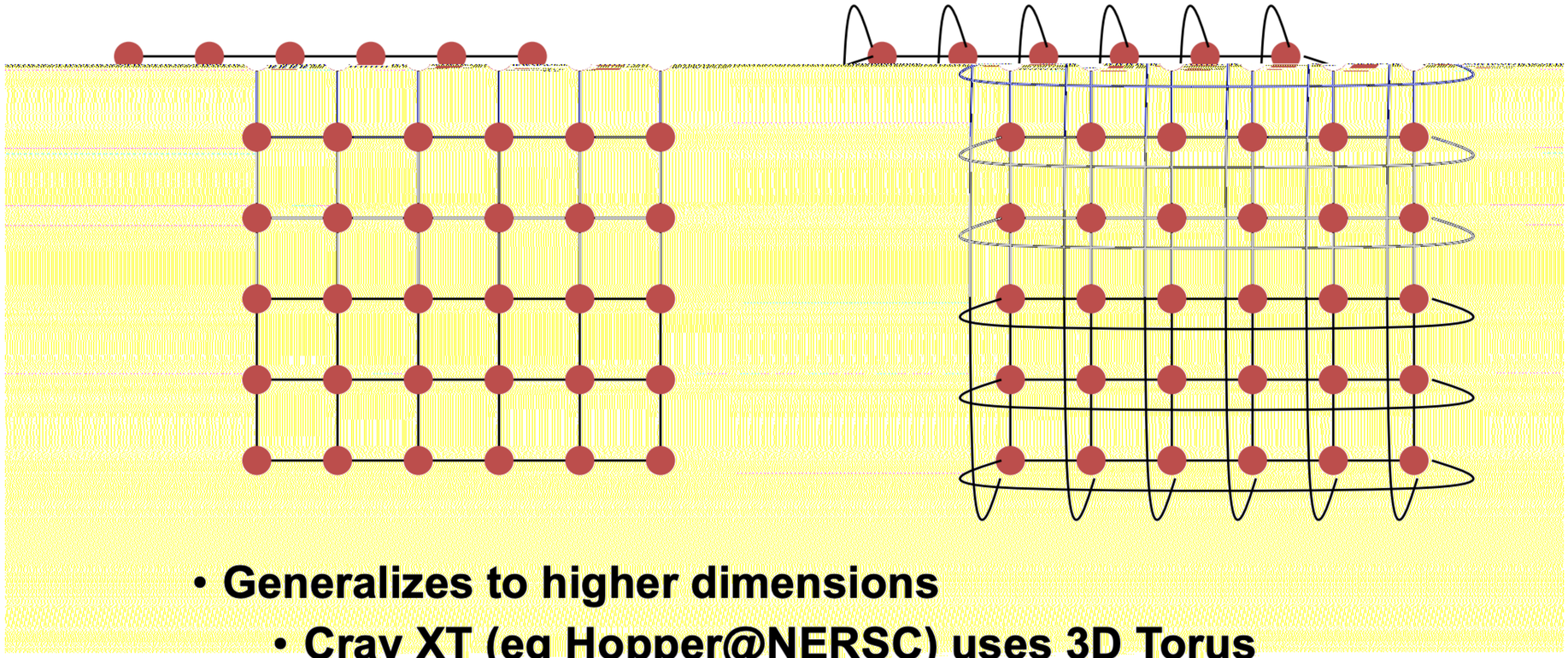
Meshes and Tori

Two dimensional mesh

- **Diameter = 2 * (sqrt(n) – 1)**
- **Bisection bandwidth = sqrt(n)**

Two dimensional torus

- **Diameter = sqrt(n)**
- **Bisection bandwidth = 2* sqrt(n)**

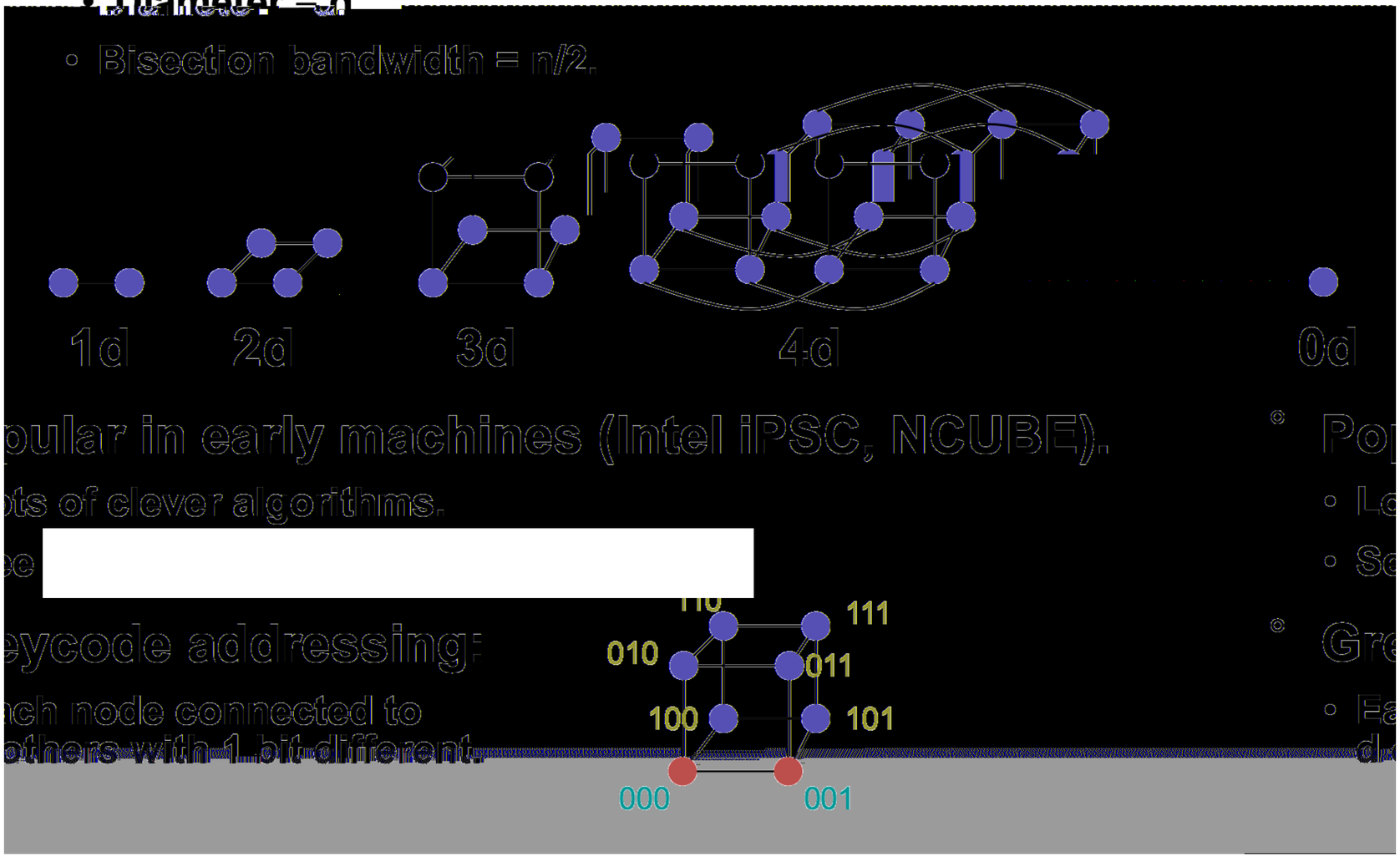


- **Generalizes to higher dimensions**
  - **Cray XT (eg Hopper@NERSC) uses 3D Torus**
- **Natural for algorithms that work with 2D and/or 3D arrays (mat**

mul)

Hypercubes

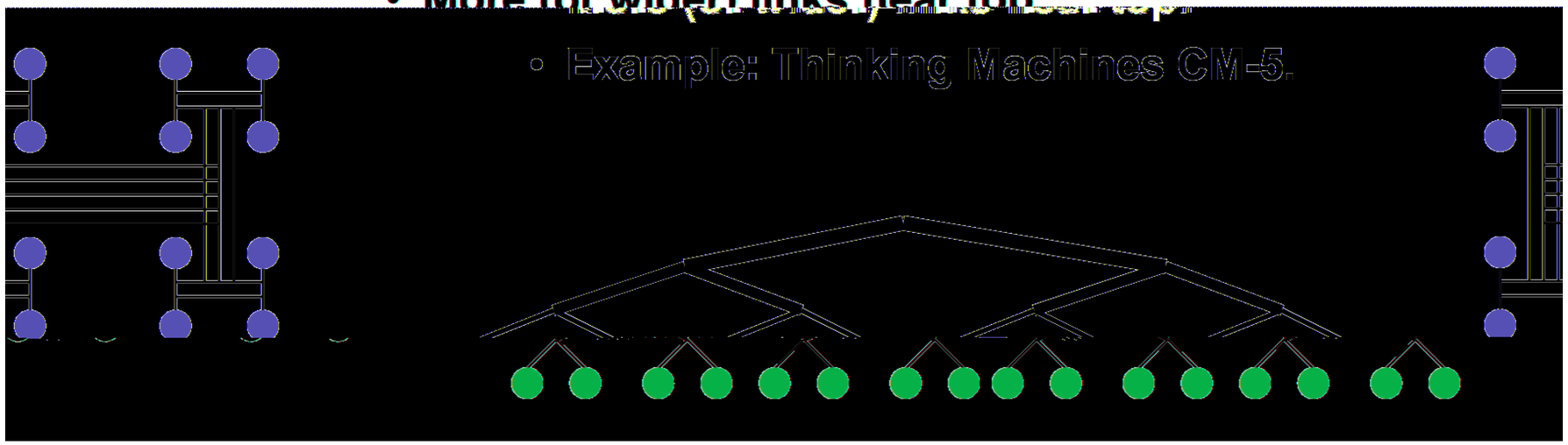
- **Number of nodes n = 2^d for dimension d.**
  - Diameter = d
  - Bisection bandwidth = n/2.



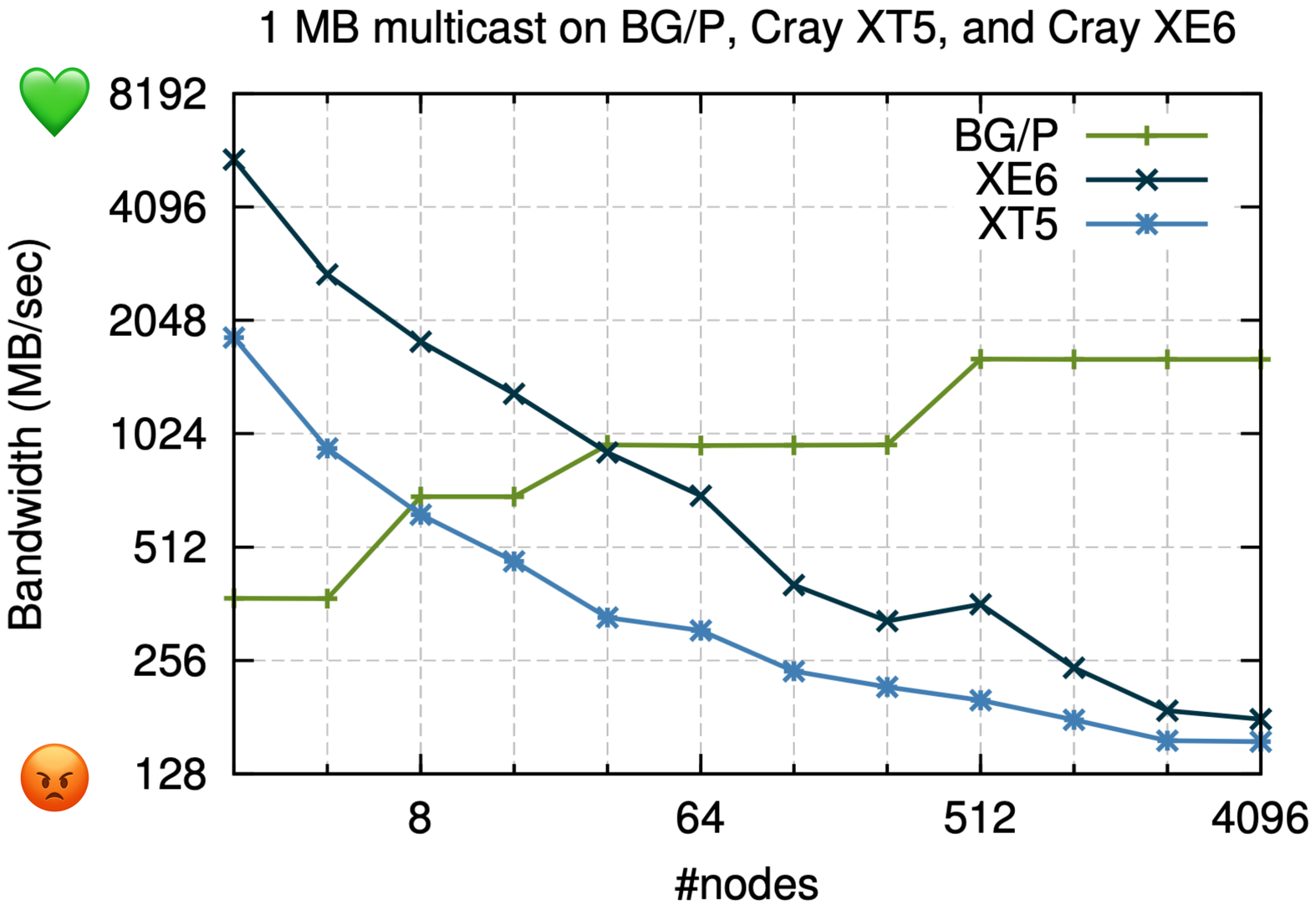
From UC Berkeley CS267

Trees

- Diameter = log n.
- Bisection bandwidth = 1.
- Easy layout as planar graph.
- Many tree algorithms (e.g., summation).
- Fat trees avoid bisection bandwidth problem:
  - More (or wider) links near top.
  - Example: Thinking Machines CM-5.



Performance Effect of Topology



From “Improving communication performance in dense linear algebra via topology aware collectives”, Solomonik, Bhatele, and Demmel, SC ‘11.

Why so many topologies?

Different systems have different needs depending on the **size of the system** (data center vs. NIC)

Physical constraints - **innovations in HW** enable previously infeasible technologies

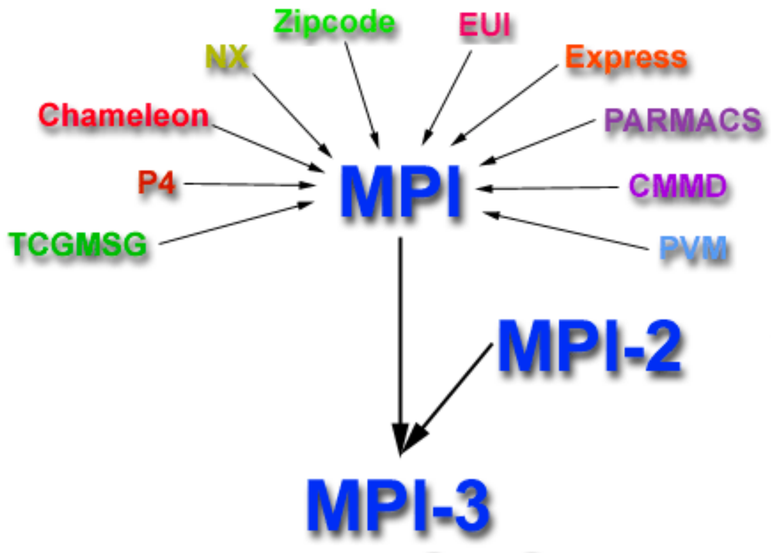
Two recent technological changes:

- Higher **radix** (number of ports supported) switches economical, which is really a consequence of Moore's law
- **Fiber optic** is feasible -> distance doesn't matter

Programming Distributed-Memory Machines with Message Passing

Message Passing Interface (MPI)

- In the 1980s-early 90s, there used to be many software tools for writing distributed-memory programs.
- 1994 - MPI 1.0 is released - works on distributed and shared memory
- Standards are needed to write portable code.



Message Passing Libraries

All **communication, synchronization require subroutine calls**

- **No shared variables**
- Program run on a single processor just like any uniprocessor program, except for calls to message passing library

Subroutines for

- **Communication**
  - Pairwise or point-to-point: Send and Receive
  - Collectives all processor get together to
    Move data: Broadcast, Scatter/gather
    Compute and move: sum, product, max, prefix sum, … of data on many processors
- **Synchronization** (barrier)
- **Enquiries** - How many processes? Which one am I? Any messages waiting?

Compile/Run MPI

In your code:

```
#include <mpi.h>

rc = MPI_XXXX(parameter, …)
```

To compile:

```
$ mpicc mpi_hello_world.c -o mpi_hello_world
```

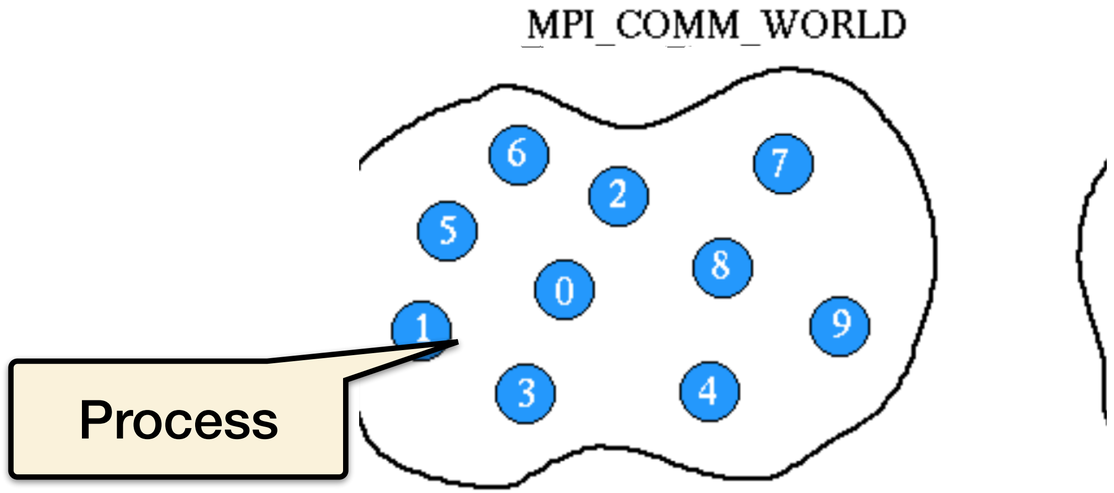
To run:

```
$ srun mpi_hello_world
```

(The default implementation of MPI on PACE ICE is mvapich2.
Different implementations (e.g., OpenMPI) may have slightly different usage)

MPI Communicator

- An **MPI communicator**: a "communication universe" for a group of processes
- `MPI_COMM_WORLD` – name of the default MPI communicator, i.e., the collection of all processes
- Each process in a communicator is identified by its **rank**
- Almost every MPI command needs to provide **a communicator as input argument**



MPI Process Rank

- Each process has a unique **rank**, i.e. an integer identifier, within a communicator
- The rank value is between `0` and `#procs-1`
- The rank value is used to distinguish one process from another
- Commands `MPI_Comm_size` & `MPI_Comm_rank` are very useful

Example:

```
int size, my_rank;

MPI_Comm_size (MPI_COMM_WORLD, &size);
MPI_Comm_rank (MPI_COMM_WORLD, &my_rank);

if (my_rank==0) { ... }
```

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len);

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

What is my name?

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len);

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size);

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

What is my name?

Print name, rank, total processor count

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len);

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size);

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

What is my name?

Print name, rank, total processor count

Finalize after we are done

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

Parallel Execution Model

The same MPI program is executed concurrently **on each process**

Process 0

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number
of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of
the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the
name of the processor

  printf("Hello world from processor %s, rank %d out of %d
processors\n",
         processor_name, world_rank, world_size); // Print hello
world message

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

Process 1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number
of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of
the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the
name of the processor

  printf("Hello world from processor %s, rank %d out of %d
processors\n",
         processor_name, world_rank, world_size); // Print hello
world message

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

...

Process P-1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number
of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of
the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the
name of the processor

  printf("Hello world from processor %s, rank %d out of %d
processors\n",
         processor_name, world_rank, world_size); // Print hello
world message

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

MPI Hello World Example Output

How to run it:

```
$ salloc -N2 --ntasks-per-node=4 -t1:00:00
salloc: Pending job allocation 1471
salloc: job 1471 queued and waiting for resources
$ srun mpi_hello_world
```

2 nodes * 4 processes per node = 8

Example output:

Order not guaranteed

```
Hello world from processor atl0, rank 0 out of 8 processors
Hello world from processor atl0, rank 2 out of 8 processors
Hello world from processor atl0, rank 3 out of 8 processors
Hello world from processor atl1, rank 4 out of 8 processors
Hello world from processor atl1, rank 7 out of 8 processors
Hello world from processor atl0, rank 1 out of 8 processors
Hello world from processor atl1, rank 5 out of 8 processors
Hello world from processor atl1, rank 6 out of 8 processors
```

https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042096#mpi-jobs

Synchronization

- Many parallel algorithms require that **no process proceeds before all the processes** have reached the same state at certain points of a program.

- Explicit synchronization :

```
int MPI_Barrier (MPI_Comm comm)
```

- Implicit synchronization through use of e.g. pairs of `MPI_Send` and `MPI_Recv`.

- Ask yourself the following question: "*If Process 1 progresses 100 times faster than Process 2, will the final result still be correct?*"

Example: Ordered Output

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

  for(int i = 0; i < world_size; i++) {
    MPI_Barrier(MPI_COMM_WORLD);
    if (i == world_rank) {
      printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size); // Print hello world message
      fflush(stdout);
    }
  }

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Iterate over processor count

Example: Ordered Output

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

  for(int i = 0; i < world_size; i++) {
    MPI_Barrier(MPI_COMM_WORLD);
    if (i == world_rank) {
      printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size); // Print hello world message
      fflush(stdout);
    }
  }

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Iterate over processor count

Synchronization

Example: Ordered Output

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

  for(int i = 0; i < world_size; i++) {
    MPI_Barrier(MPI_COMM_WORLD);
    if (i == world_rank) {
      printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size); // Print hello world message
      fflush(stdout);
    }
  }

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Iterate over processor count

Synchronization

Only print if we are on our rank

Example: Ordered Output

The processes synchronize between themselves P times:

Process 0

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

    for(int i = 0; i < size; i++) {
         MPI_Barrier(MPI_COMM_WORLD);
         if (i == world_rank) {
           printf("Hello world from processor %s, rank %d out of
         %d processors\n",
             processor_name, world_rank, world_size); // Print
    hello world message
              fflush(stdout);
         }
    }

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

Process 1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

    for(int i = 0; i < size; i++) {
         MPI_Barrier(MPI_COMM_WORLD);
         if (i == world_rank) {
           printf("Hello world from processor %s, rank %d out of
         %d processors\n",
             processor_name, world_rank, world_size); // Print
    hello world message
              fflush(stdout);
         }
    }

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

...

Process P-1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

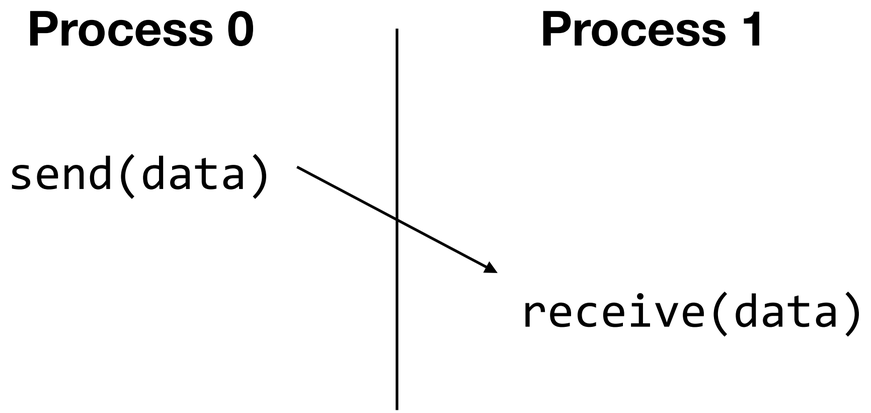
    for(int i = 0; i < size; i++) {
         MPI_Barrier(MPI_COMM_WORLD);
         if (i == world_rank) {
           printf("Hello world from processor %s, rank %d out of
         %d processors\n",
             processor_name, world_rank, world_size); // Print
    hello world message
              fflush(stdout);
         }
    }

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

Output:

```
Hello world from processor atl0, rank 0 out of 8 processors
Hello world from processor atl0, rank 1 out of 8 processors
Hello world from processor atl0, rank 2 out of 8 processors
Hello world from processor atl0, rank 3 out of 8 processors
Hello world from processor atl1, rank 4 out of 8 processors
Hello world from processor atl1, rank 5 out of 8 processors
Hello world from processor atl1, rank 6 out of 8 processors
Hello world from processor atl1, rank 7 out of 8 processors
```

MPI Basic Send/Receive



Things that need specifying:

- How will “data” be described?
- How will processes be identified?
- How will the receiver recognize/screen messages?
- What will it mean for these operations to complete?

MPI Datatypes

- The data in a message to send or receive is described by a triple (address, count, datatype), where

- An MPI datatype is recursively defined as:
  - **predefined**, corresponding to a data type from the language (e.g., `MPI_INT, MPI_DOUBLE`)
  - a contiguous array of MPI datatypes
  - a strided block of datatypes
  - an indexed array of blocks of datatypes
  - an arbitrary structure of datatypes

- There are MPI functions to construct custom datatypes, in particular ones for subarrays
- May hurt performance if datatypes are complex

The Simplest MPI Send Command

```
int MPI_Send(void *buf, int count,
             MPI_Datatype datatype,
             int dest, int tag,
             MPI_Comm comm)
```

Returns only when communication is finished

This **blocking send function** returns when the data has been delivered to the system and the buffer can be reused. The message may not have been received by the destination process.

The Simplest MPI Receive Command

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

- This **blocking receive function** waits until a matching message is received from the system so that the buffer contains the incoming message.
- Match of data type, source process (or `MPI_ANY_SOURCE`), message tag (or `MPI_ANY_TAG`).
- Receiving fewer `datatype` elements than count is ok, but receiving more is an error.

Assuming 2 processes

A Simple MPI Send/Receive Program

```
#include “mpi.h”
#include <stdio.h>

int main( int argc, char *argv[]) {
  int rank, buf;
  MPI_Status status;
  MPI_Init(&argv, &argc);
  MPI_Comm_rank( MPI_COMM_WORLD, &rank );

  /* Process 0 sends and Process 1 receives
  if (rank == 0) {
    buf = 123456;
    MPI_Send( &buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
  } else if (rank == 1) {
    MPI_Recv( &buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status );
    printf( “Received %d\n ”, buf );
  }
  MPI_Finalize();
return 0;
}
```

```
int MPI_Send(void *buf, int count,
             MPI_Datatype datatype,
             int dest, int tag,
             MPI_Comm comm)
```

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

MPI Tags

```
int MPI_Send(void *buf, int count,
             MPI_Datatype datatype,
             int dest, int tag,
             MPI_Comm comm)
```

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

Messages are sent with an accompanying user-defined integer tag, to assist the receiving process in identifying the message.

Messages can be screened at the receiving end by specifying a specific tag, or not screened by specifying `MPI_ANY_TAG` as the tag in a receive.

https://mpitutorial.com/tutorials/dynamic-receiving-with-mpi-probe-and-mpi-status/

MPI Status

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

In C, the `MPI_Status` is a structure that contains at least 3 attributes: `MPI_SOURCE`, `MPI_TAG` and `MPI_ERROR`.

- `MPI_SOURCE` contains the rank of the process that sent the message.
- `MPI_TAG` contains the tag of the message received.
- `MPI_ERROR` contains the error code of the receive operation.

https://mpitutorial.com/tutorials/dynamic-receiving-with-mpi-probe-and-mpi-status/

Querying `MPI_Status`

```
MPI_Get_count(
    MPI_Status* status,
    MPI_Datatype datatype,
    int* count)
```

In `MPI_Get_count`, the user passes the `MPI_Status` structure, the `datatype` of the message, and `count` is returned. The `count` variable is the total number of `datatype` elements that were received.

`MPI_Recv` can take `MPI_ANY_SOURCE` for the rank of the sender and `MPI_ANY_TAG` for the tag, so `MPI_Status` would be the only way to find the sender and tag in that case.

Example of Querying `MPI_Status`

```
const int MAX_NUMBERS = 100;
int numbers[MAX_NUMBERS];
int number_amount;
if (world_rank == 0) {
    // Pick a random amount of integers to send to process one
    srand(time(NULL));
    number_amount = (rand() / (float)RAND_MAX) * MAX_NUMBERS;

    // Send the amount of integers to process one
    MPI_Send(numbers, number_amount, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("0 sent %d numbers to 1\n", number_amount);
}
…
```

https://mpitutorial.com/tutorials/dynamic-receiving-with-mpi-probe-and-mpi-status/

Example of Querying MPI_Status

```
…
} else if (world_rank == 1) {
    MPI_Status status;
    // Receive at most MAX_NUMBERS from process zero
    MPI_Recv(numbers, MAX_NUMBERS, MPI_INT, 0, 0, MPI_COMM_WORLD,
             &status);

    // After receiving the message, check the status to determine
    // how many numbers were actually received
    MPI_Get_count(&status, MPI_INT, &number_amount);

    // Print off the amount of numbers, and also print additional
    // information in the status object
    printf("1 received %d numbers from 0. Message source = %d, "
           "tag = %d\n",
           number_amount, status.MPI_SOURCE, status.MPI_TAG);
}
```

Output Example:

```
0 sent 93 numbers to 1
1 dynamically received 93 numbers from 0
```

MPI_Probe

```
MPI_Probe(
    int source,
    int tag,
    MPI_Comm comm,
    MPI_Status* status)
```

- `MPI_Probe` will block for a message with a matching tag and sender. When it receives it, it will fill the status structure with information. Then the user can use `MPI_Recv` to receive the actual message.
- `MPI_Probe` forms the basis of many **dynamic MPI applications**. For example, manager/worker programs will often make heavy use of `MPI_Probe` when exchanging variable-sized worker messages.

MPI_Probe Example

```
…
} else if (world_rank == 1) {
    MPI_Status status;
    // Probe for an incoming message from process zero
    MPI_Probe(0, 0, MPI_COMM_WORLD, &status);

    // When probe returns, the status object has the size and other
    // attributes of the incoming message. Get the message size
    MPI_Get_count(&status, MPI_INT, &number_amount);

    // Allocate a buffer to hold the incoming numbers
    int* number_buf = (int*)malloc(sizeof(int) * number_amount);

    // Now receive the message with the allocated buffer
    MPI_Recv(number_buf, number_amount, MPI_INT, 0, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("1 dynamically received %d numbers from 0.\n",
           number_amount);
    free(number_buf);
}
```

https://mpitutorial.com/tutorials/dynamic-receiving-with-mpi-probe-and-mpi-status/

MPI Can Be Simple

Claim: Most MPI applications can be written with only 6 functions (although which 6 may differ)

Using point-to-point:

- `MPI_INIT`
- `MPI_FINALIZE`
- `MPI_COMM_SIZE`
- `MPI_COMM_RANK`
- `MPI_SEND`
- `MPI_RECEIVE`

Using collectives:

- `MPI_INIT`
- `MPI_FINALIZE`
- `MPI_COMM_SIZE`
- `MPI_COMM_RANK`
- `MPI_BCAST`
- `MPI_REDUCE`

You may use more for convenience or performance.

But is that small subset the practical usage?

35 **What aspects of the MPI standard do you use in your application in its current form?** * *(multiple)*

36 **What aspects of the MPI standard appear in performance-critical sections of your current application?** * *(multiple)*

37 **What aspects of the MPI standard do you anticipate using in the "exascale" version of your application?** * *(multiple)*

	Q35: Current Usage			Q37: Exascale Usage			Q36: Performance Critical		
Responses	**AD**	**ST**	**Overall**	**AD**	**ST**	**Overall**	**AD**	**ST**	**Overall**
Point-to-point communications	96%	79%	88%	89%	71%	80%	93%	75%	84%
MPI derived datatypes	25%	21%	23%	21%	21%	21%	14%	7%	11%
Collective communications	86%	75%	80%	96%	68%	82%	64%	64%	64%
Neighbor collective communications	14%	14%	14%	32%	25%	29%	7%	11%	9%
Communicators and group management	68%	54%	61%	61%	50%	55%	29%	7%	18%
Process topologies	14%	7%	11%	32%	11%	21%	4%	4%	4%
RMA (one-sided communications)	36%	7%	21%	50%	36%	43%	21%	7%	14%
RMA shared windows	18%	7%	12%	21%	18%	20%	7%	7%	7%
MPI I/O (called directly)	25%	18%	21%	21%	18%	20%	4%	7%	5%
MPI I/O (called through a third-party library)	32%	21%	27%	36%	25%	30%	7%	11%	9%
MPI profiling interface	11%	0%	14%	11%	21%	16%	0%	4%	2%

From UC Berkeley CS267 https://info.ornl.gov/sites/publications/Files/Pub108588.pdf

But is that small subset the practical usage?

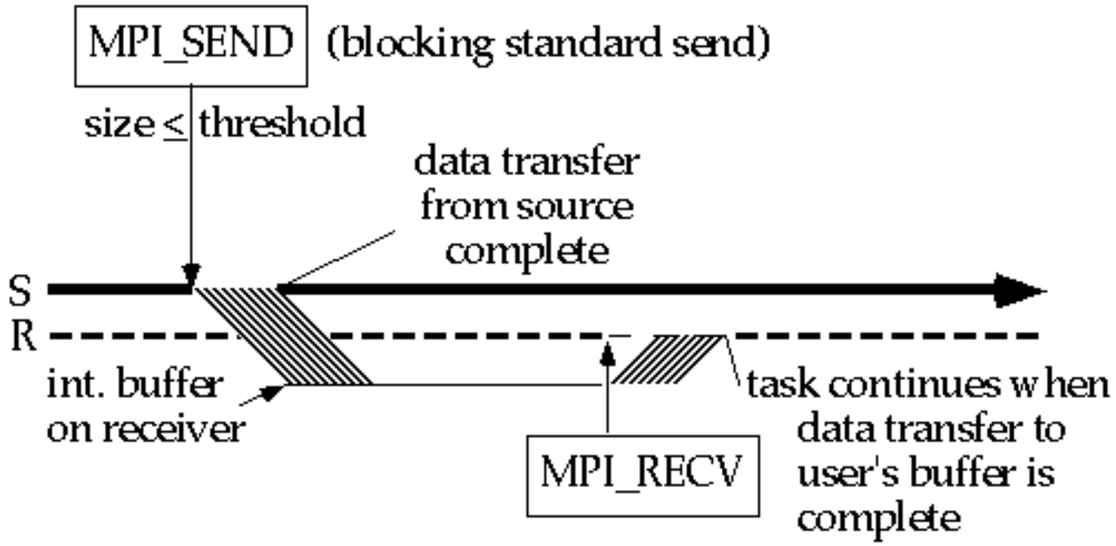
No.	Question and Responses	AD	ST	Overall
38	**What is the dominant communication in your application?** Check all that apply, recognizing that many applications have different communication patterns in different phases. *(multiple)*			
	Each process talks to (almost) every other process	25%	21%	23%
	Processes communicate in fixed "neighborhoods" of limited size	46%	46%	46%
	Processes communicate in "neighborhoods" of limited size that may change in different phases of the application or evolve over the course of a run	42%	36%	39%
	Communication is largely irregular	18%	21%	20%
39	**Can your application take advantage of non-blocking point-to-point operations...** *(multiple)*			
	To overlap communication with computation?	89%	71%	80%
	To allow asynchronous progress?	64%	64%	64%
	To allow event-based programming?	43%	29%	36%
40	**Can your application take advantage of non-blocking collective operations...** *(multiple)*			
	To overlap communication with computation?	71%	46%	59%
	To allow asynchronous progress?	43%	29%	36%
	To allow event-based programming?	25%	14%	20%

Blocking and Non-Blocking Communication

So far we have been using blocking communication:

- MPI_Recv does not complete until the buffer is full (available for use).
- MPI_Send does not complete until the buffer is empty (available for use).

Completion depends on size of message and amount of system buffering.



https://www.codingame.com/playgrounds/47058/have-fun-with-mpi-in-c/blocking-communication

Non-Blocking Send/Receive

Non-blocking operations **return (immediately)** “request handles” that can be tested and waited on:

- `MPI_Request request;`
- `MPI_Status status;`
- `MPI_Isend(…);`
- `MPI_Irecv(…);`
- `MPI_Wait(&request, &status);` (each request must be Waited on)

One can also test without waiting:

```
MPI_Test(&request, &flag, &status);
```

Accessing the data buffer without waiting is undefined

MPI_Isend

```
int MPI_Isend(const void* buffer,
              int count,
              MPI_Datatype datatype,
              int recipient,
              int tag,
              MPI_Comm communicator,
              MPI_Request* request);
```

Handle on non-blocking operation (used later to check for completion)

- `MPI_Isend` is the standard **non-blocking send** (I stands for immediate return).
- The user must not attempt to reuse the buffer after `MPI_Isend` returns without explicitly **checking for completion**.

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

`MPI_Wait` and `MPI_Test`

```
int MPI_Wait(MPI_Request* request,
             MPI_Status* status);
```

`MPI_Wait` **waits** for a non-blocking operation to complete and will block until the underlying operation is done.

```
int MPI_Test(MPI_Request* request,
             int* flag,
             MPI_Status* status);
```

`MPI_Test` checks if a non-blocking operation is complete at a given time.

It will not wait for the underlying non-blocking operation to complete.

https://rookiehpc.org/mpi/docs/mpi_wait/index.html
https://rookiehpc.org/mpi/docs/mpi_test/index.html

Two processes

MPI_Isend Example - Sender

```
   switch(my_rank)
    {
        case SENDER:
        {
            int buffer_sent = 12345;
            MPI_Request request;
            printf("MPI process %d sends value %d.\n", my_rank, buffer_sent);
            MPI_Isend(&buffer_sent, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Isend completes
            // <...>

            // Let's wait for the MPI_Isend to complete before progressing
further.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            break;
        }
        // RECEIVER CASE ON NEXT SLIDE
    }
```

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

Two processes

MPI_Isend Example - Sender

```
   switch(my_rank)
    {
        case SENDER:
        {
            int buffer_sent = 12345;
            MPI_Request request;
            printf("MPI process %d sends value %d.\n", my_rank, buffer_sent);
            MPI_Isend(&buffer_sent, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Isend completes
            // <...>

            // Let's wait for the MPI_Isend to complete before progressing
further.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            break;
        }
        // RECEIVER CASE ON NEXT SLIDE
    }
```

MPI process 0 sends value 12345

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

Two processes

MPI_Isend Example - Sender

```
   switch(my_rank)
    {
        case SENDER:
        {
            int buffer_sent = 12345;
            MPI_Request request;
            printf("MPI process %d sends value %d.\n", my_rank, buffer_sent);
            MPI_Isend(&buffer_sent, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Isend completes
            // <...>

            // Let's wait for the MPI_Isend to complete before progressing
further.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            break;
        }
        // RECEIVER CASE ON NEXT SLIDE
    }
```

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

Two processes

MPI_Isend Example - Receiver

```
switch(my_rank)
 {
      // CASE SENDER FROM BEFORE NOT SHOWN
     case RECEIVER:
     {
         int received;
         MPI_Recv(&received, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
         printf("MPI process %d received value: %d.\n", my_rank, received);
         break;
     }
 }
```

Does not need to be a special recv

MPI process 1 received value 12345

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

MPI_Irecv

```
int MPI_Irecv(void* buffer,
              int count,
              MPI_Datatype datatype,
              int sender,
              int tag,
              MPI_Comm communicator,
              MPI_Request* request);
```

Handle on non-blocking operation (used later to check for completion)

- `MPI_Irecv` is the standard **non-blocking receive** (I stands for immediate return).
- To know whether the message has been received, you must use `MPI_Wait` or `MPI_test` on the `MPI_Request`.

https://rookiehpc.org/mpi/docs/mpi_irecv/index.html

Summary

- There are many different network topologies, each with their own strengths and weaknesses.
- The main performance metrics of a network are **latency and bandwidth**.
- MPI defines a standard for programming distributed-memory machines.
- The core functionality for message passing is **send and receive** (and variants)

MPI References

- MPI Standard: https://www.mpi-forum.org/
- Other information: https://www.mcs.anl.gov/research/projects/mpi/index.htm
- LLNL tutorial: https://hpc-tutorials.llnl.gov/mpi/
- https://mpitutorial.com/

Backup Slides & More Examples

More MPI Send/Receive Examples

```
int ping_pong_count = 0;
int partner_rank = (world_rank + 1) % 2;
while (ping_pong_count < PING_PONG_LIMIT) {
    if (world_rank == ping_pong_count % 2) {
        // Increment the ping pong count before you send it
        ping_pong_count++;
        MPI_Send(&ping_pong_count, 1, MPI_INT, partner_rank, 0,
                 MPI_COMM_WORLD);
        printf("%d sent and incremented ping_pong_count "
               "%d to %d\n", world_rank, ping_pong_count,
               partner_rank);
    } else {
        MPI_Recv(&ping_pong_count, 1, MPI_INT, partner_rank, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("%d received ping_pong_count %d from %d\n",
               world_rank, ping_pong_count, partner_rank);
    }
}
```

https://mpitutorial.com/tutorials/mpi-send-and-receive/

MPI Ping Pong Output

```
0 sent and incremented ping_pong_count 1 to 1
0 received ping_pong_count 2 from 1
0 sent and incremented ping_pong_count 3 to 1
0 received ping_pong_count 4 from 1
0 sent and incremented ping_pong_count 5 to 1
0 received ping_pong_count 6 from 1
0 sent and incremented ping_pong_count 7 to 1
0 received ping_pong_count 8 from 1
0 sent and incremented ping_pong_count 9 to 1
0 received ping_pong_count 10 from 1
1 received ping_pong_count 1 from 0
1 sent and incremented ping_pong_count 2 to 0
1 received ping_pong_count 3 from 0
1 sent and incremented ping_pong_count 4 to 0
1 received ping_pong_count 5 from 0
1 sent and incremented ping_pong_count 6 to 0
1 received ping_pong_count 7 from 0
1 sent and incremented ping_pong_count 8 to 0
1 received ping_pong_count 9 from 0
1 sent and incremented ping_pong_count 10 to 0
```

MPI Ring Send/Receive Example

```
int token;
if (world_rank != 0) {
    MPI_Recv(&token, 1, MPI_INT, world_rank - 1, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process %d received token %d from process %d\n",
           world_rank, token, world_rank - 1);
} else {
    // Set the token's value if you are process 0
    token = -1;
}
MPI_Send(&token, 1, MPI_INT, (world_rank + 1) % world_size,
         0, MPI_COMM_WORLD);

// Now process 0 can receive from the last process.
if (world_rank == 0) {
    MPI_Recv(&token, 1, MPI_INT, world_size - 1, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process %d received token %d from process %d\n",
           world_rank, token, world_size - 1);
}
```

https://mpitutorial.com/tutorials/mpi-send-and-receive/

MPI Ring Send/Receive Output

```
Process 1 received token -1 from process 0
Process 2 received token -1 from process 1
Process 3 received token -1 from process 2
Process 4 received token -1 from process 3
Process 0 received token -1 from process 4
```